Chapitre 1 : Rappels sur la Représentation et la Nature des Données Multimédia

Table des matières

Introduction 1
1 Communication Multimédia 1
1.1 Définition 1
1.2 Caractéristiques 1
1.3 éléments d'un Système Multimédia 2
1.4 Types Multimédia 2
2 Nature des données multimédias 3
2.1 Son 3
2.2 Image 9
2.3 Vidéo 15
2.4 Haptique 20
Conclusion 26

Table des figures

1 Éléments d'un Système Multimédia 2
2 chronogrammes d'un signal en temps discret et continu + signal numérisé 4
3 Enveloppe ADSR d'un son 4
4 Théoréme de Shanon 5
5 Echantillonage 6
6 Quantification 7
7 Image Vectorielle 9
8 Image Matricielle et son Zoom 10
9 Types d'image Matricielle 11
10 Image RGB , canal rouge - Vert et Bleu 12
11 Equation DCT 13
12 Zigzag 14
13 Frames Video 15
14 Image Prédictive 17
15 Image Bidirectionnelle 17
16 Haptique Kinésthésique 21
17 Haptique Tactile 21
18 Anatomie Sensitive de la main 22

Liste des tableaux

1 Comparaison compression avec perte vs sans perte 8
2 Exemple Image 14

Introduction

À l'heure où l'information est omniprésente et l'attention de plus en plus fragmentée, communiquer ne suffit plus, il faut marquer les esprits. Si la communication traditionnelle reposait sur un seul canal, la révolution numérique a fait émerger un nouveau paradigme. C'est dans ce contexte de mutation que s'inscrit la communication multimédia : une approche globale qui fusionne texte, image, son, vidéo et interactivité pour créer une expérience cohérente, immersive et engageante. Comment cette convergence redéfinit-elle notre manière de transmettre un message . La communication multimédia repose sur trois familles de données aux natures physiques fondamentalement différentes : le son (onde de pression longitudinale), l'image et la vidéo (radiance spatiale échantillonnée), et l'haptique (interaction bidirectionnelle force/position/tactile). Chaque famille exige une chaîne de numérisation spécifique – échantillonnage, quantification, codage – puis des codecs perceptuels (psychoacoustique pour l'audio, DCT et compensation de mouvement pour la vidéo, deadband perceptuel pour l'haptique) et enfin des protocoles de transport temps-réel (RTP/RTCP, SIP, RTSP, IEEE 1918.1.1). Ce premier chapitre détaille ces représentations

1 Communication Multimédia

Le mot multimédia combine Multi et Media qui veut dire l'intégration sur un même support numérique de plusieurs types d'informations qui utilisent des sens différents. Une donnée multimédia a 3 caractéristiques qui la rendent complexe :

- Volume énorme : 1 image HD non compressée = 6 Mo, 1 minute de vidéo 4K = 5 Go
- Dimension temporelle : Le texte et l'image sont statiques. L'audio et la vidéo sont dynamiques, ils ont besoin de synchronisation.
- Hétérogénéité : On doit mélanger des données qui n'ont rien à voir entre elles.

1.1 Définition

La Communication Multimédia désigne l'intégration et la manipulation de différentes formes de médias (texte, images fixes, sons, vidéos, animations, éléments interactifs, données haptiques) de maniére synchronisée et interactive pour transmettre un message ou interagir avec un utilisateur. Elle vise à enrichir l'expérience de communication en stimulant plusieurs sens simultanément ou séquentiellement, offrant ainsi une compréhension plus profonde, une meilleure rétention de l'information et une plus grande implication de l'audience.

1.2 Caractéristiques

Les caractéristiques principales de la communication multimédia sont :

- Multimodalité :Utilisation simultanée ou séquentielle de plusieurs modalités sensorielles.
- Interactivité :Possibilité pour l'utilisateur d'influencer le déroulement de la communication (navigation, choix de contenu, saisie de données).
- Numérisation :Tous les médias sont encodés et traités numériquement, permettant leur intégration et leur diffusion via des supports numériques.
- Intégration :Cohésion entre les différents médias pour former un tout signifiant.

1.3 éléments d'un Système Multimédia

Le système multimédia regroupe 6 éléments indispensables : Matériel + Logiciels + Données + Réseau + Interaction, régis par des Normes.



FIGURE 1 : Éléments d'un Système Multimédia

1. **Matériel / Hardware :** Capture : caméra, micro, scanner, clavier, Traitement : CPU, GPU, carte son, carte graphique
2. **Stockage** : disque dur, SSD, RAM, Cloud, Restitution : écran, haut-parleur, casque VR, imprimante
3. **Logiciels / Software :** Système d'exploitation (Windows, Android) , Logiciels d'édition : Photoshop (image), Audacity (son), Premiere (vidéo) Codecs : pour encoder/décoder → H.264 pour vidéo, MP3 pour son, JPEG pour image, Lecteurs multimédia : VLC, etc.
4. **Données médias :** Texte : .txt, .pdf, .doc, Image : Raster = JPEG, PNG / Vectorielle = SVG Son : échantillonné = WAV, MP3, Vidéo / Animation : MP4, GIF, 3D
5. **Réseau et Transmission Protocoles :** TCP/IP, HTTP (port 80), RTP (5004), HLS, Débit, bande passante, latence Modes : streaming temps réel ou téléchargement
6. **Interaction IHM et Hypermédia :** IHM : Interface Homme-Machine (écran tactile, souris) Hypermédia : liens, boutons, menus qui permettent de naviguer , Interactivité : l'utilisateur clique, fait glisser, parle
7. **Normes et Formats :** Pour que tout soit compatible : Vidéo : MPEG-4, H.264 / HEVC H.265, Image : JPEG, PNG, Audio : MP3, AAC, WAV

1.4 Types Multimédia

1. **Médias DISCRETS** : Indépendant du temps Exemples : Texte, Image fixe JPEG/PNG, graphique vectoriel ; Taille fixe, affichage instantané, Pas de débit, pas de synchronisation, pas de latence, Tu ouvres et c'est tout
2. **Médias CONTINUS** : Dépendant du temps Exemples : Son MP3, Vidéo MP4, Animation Flux qui évolue dans le temps, Besoin de débit constant, buffer, gestion latence/jitter, Synchronisation obligatoire : son + image doivent être alignés, Streaming temps réel obligatoire

Flux vs Fichier

FICHIER = Téléchargement Stockage complet sur disque, taille connue dès le départ, Accès aléatoire possible (avance/recul instantané) Lecture après téléchargement complet ou progressive download, Pas de

contrainte temps réel alors utilisation hors ligne possible Exemples : MP4 local, JPEG, MP3 téléchargé, PDF ,Schéma : Serveur envoie TOUT → puis client lit

FLUX = Streaming / direct Données en continu, taille et durée inconnue au départ ,Lecture en temps réel PENDANT la réception, Buffer (mémoire tampon) pour lisser jitter/retards ; Débit constant nécessaire + adaptation ABR (la qualité s'adapte) Exemples : YouTube HLS/DASH en TCP 80/443, visio RTP UDP 5004 Zoom/Teams, Radio Internet live

le Fichier est idéal pour contenu statique hors ligne, le Flux est idéal pour direct, live, visio temps réel.

2 Nature des données multimédias

Tous les médias sont numérisés (convertis en binaire) , pour être stockés traités et transmis sur le même réseau .

2.1 Son

Le son n'est pas une onde électromagnétique comme la lumière. C'est une onde mécanique longitudinale : une variation de pression de l'air.

Le son est une onde de pression acoustique longitudinale se propageant dans un milieu élastique. En communication multimédia, on le modélise comme un signal continu x(t) à bande limitée, caractérisé par sa fréquence, son amplitude et sa phase.

Nature Physique

s(t) est le signal audio. Son graphique (temps en X, amplitude en Y) s'appelle le chronogramme. Le son est une donnée temporelle unidimensionnelle, dont la qualité dépend de la précision de la discrétisation du signal électrique original. On distingue trois paramètres physiques :

- Fréquence (Hz) : Nombre de vibrations par seconde.C'est ce qui donne la hauteur (son est grave ou aigu : Grave $<$ 300 Hz, Aigu $>$ 4000 Hz

  L' Oreille humaine perçoit de 20 Hz à 20 KHz avec une dynamique de ~120dB

  $f = \frac{1}{T}$ où T est la période , durée d'un cycle complet en seconde (T_e =1ms)
- Amplitude (dB) : La force de la vibration. C'est ce qui donne l'intensité / volume.
- Timbre : La forme de l'onde. C'est ce qui permet de différencier un piano d'une guitare qui jouent la même note. C'est dû aux harmoniques.
- Enveloppe ADSR : Tout son naturel : Attack (attaque rapide), Decay (chute), Sustain (tenue), Release (extinction).

Psychoacoustique - Comment l'oreille triche

L'oreille n'entend pas toutes les fréquences pareil

- Courbes isosoniques de Fletcher-Munson : L'oreille n'entend pas toutes les fréquences pareil. Elle est très sensible vers 2-4kHz (voix) et sourde dans les graves/aigus faibles.
- Masquage fréquentiel : Un son fort à 300Hz masque un son faible à 350Hz. On peut le supprimer, on ne l'entendra pas. C'est le cœur du MP3. Un son fort rend inaudible un son faible proche en fréquence.

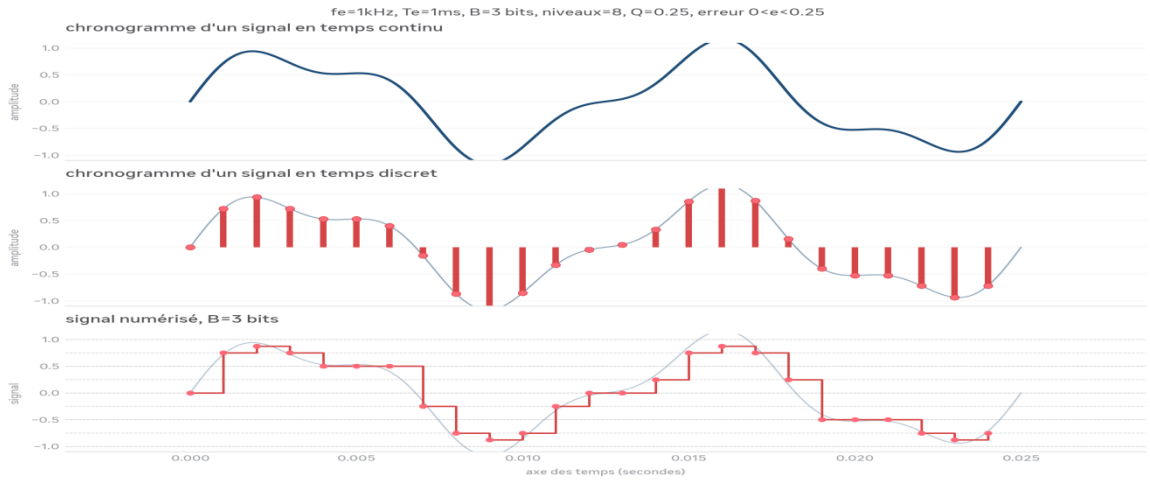


FIGURE 2 : chronogrammes d'un signal en temps discret et continu + signal numérisé

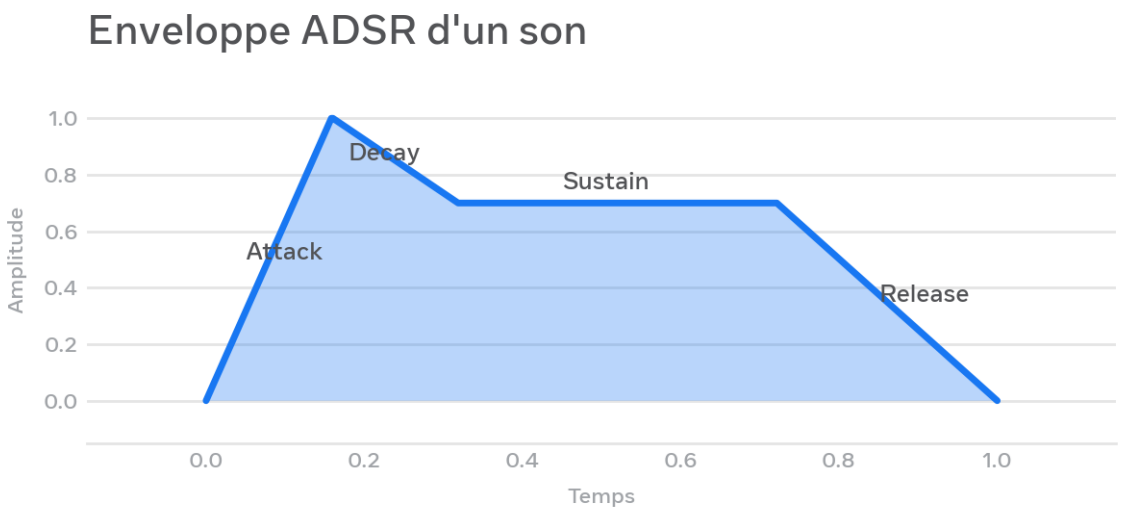


FIGURE 3 : Enveloppe ADSR d'un son

le MP3 l'utilise : Il découpe le son en 32 bandes de fréquences. Dans chaque bande, il repère le son le plus fort (le masquant). Tout ce qui est en dessous de la courbe de masquage, il ne le code pas. Gain de place énorme, 0 perte perçue.

— Masquage temporel : Un son fort masque les sons 5ms avant et 100ms après.

Post-masquage : Après un gros "BOUM" de 80dB, pendant 100-200 ms ton oreille reste sourde. Un petit son juste après est inaudible.

Pré-masquage : Plus bizarre, 5ms AVANT le gros son, tu es déjà sourd. Ton cerveau traite par paquets.

Le MP3 exploite ça : après une attaque de batterie, il n'a pas besoin de coder finement le détail pendant 100ms.

— Localisation :Comment tu sais d'où vient le son , Grâce à la différence de temps entre oreille gauche/droite (ITD).

C'est pour le son stéréo , le cerveau utilise 3 indices :

a)ITD - Interaural Time Difference : Le son arrive plus tôt dans une oreille.

ITD = $\frac{d \ \ sin}{c}$

Si le son vient de gauche, il arrive à l'oreille gauche 0.6ms avant la droite. Le cerveau calcule l'angle. Efficace pour les graves < 1500 Hz.

b) ILD - Interaural Level Difference : Ta tête fait de l'ombre. Un son venant de gauche sera plus fort à gauche qu'à droite, surtout dans les aigus > 3000 Hz qui ne contournent pas la tête.

c)HRTF - Head Related Transfer Function : Le pavillon de ton oreille déforme le son selon qu'il vient d'en haut, en bas, devant, derrière. C'est ta signature perso. C'est pour ça qu'on a le "cône de confusion" : sans bouger la tête, tu confonds devant et derrière.

De l'analogique au numérique : La chaîne de numérisation

Un micro convertit la pression en tension électrique analogique continue. L'ordinateur ne comprend que du 0et 1, donc il faut numériser.

La représentation numérique du son repose sur la numérisation d'une onde acoustique continue (analogique). Ce processus s'articule autour de deux étapes clés :

— L'échantillonnage :Consiste à prélever des échantillons de l'onde à intervalles réguliers. La fréquence d'échantillonnage (ex : 44,1 kHz pour le CD) détermine la bande passante du signal capturé.

On découpe le temps. On relève la valeur du signal tous les T_e .

$$F_e \geq 2 \times F_{max}$$

FIGURE 4 : Théoréme de Shanon

Selon le théorème de Shannon-Nyquist : On prélève la valeur de l'onde à intervalles réguliers. La fréquence d'échantillonnage F_e doit être au moins 2x la fréquence max du signal Si tu ne respectes pas ça, on aura du repliement de spectre (aliasing) : le son devient faux.

— Téléphonie : $F_{\max}$= 3400 Hz $\rightarrow$ F_e = 8 kHz
— Radio FM : F_e = 22.05 kHz
— CD Audio : F_e = 44.1 kHz (car $F_{\max}$ = 20 kHz)
— Studio pro : F_e = 48 kHz ou 96 kHz

La figure ci-dessus illustre une onde analogique sinusoïdale avec des points discrets marqués à intervalles réguliers pour représenter les échantillons.

— La quantification :Consiste à mesurerl'amplitude de chaque échantillon et à la convertir en une valeur numérique sur un nombre défini de bits (ex : 16 bits). Cela définit la dynamique sonore et le rapport signal/bruit.

On code chaque mesure sur B (N) bits :

— Nombre de niveaux = 2^N
— Pas de Quantification : Q= $\frac{V_{\max} - V_{\min}}{2^N}$ où V est la tension du signal analogique.
— Ça crée une erreur de quantification : 0 < e < Q

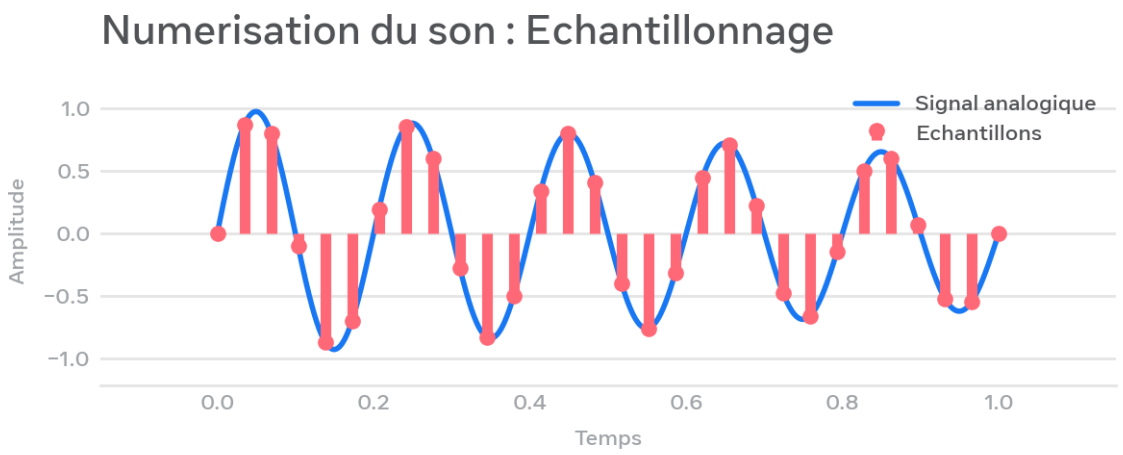


FIGURE 5 : Echantillonage

La figure ci- dessous pourrait montrer ces échantillons quantifiés, chacun aligné sur un niveau discret parmi un ensemble fini de niveaux possibles. $V_{\max}$=1 et $V_{\min}$= -1

— Sur 8 bits : 256 niveaux → bruit important
— Sur 16 bits (CD) : 65 536 niveaux
— Sur 24 bits (studio) : 16,7 millions de niveaux

On distingue 2 types :

— Uniforme (linéaire) : Niveaux équidistants. Pour CD.
— Non-uniforme (log) : Loi A (Europe) et loi (USA). On met plus de niveaux pour les petites amplitudes (la voix). 8 bits loi A = qualité de 12 bits linéaire. C'est le G.711 à 64kbit/s.

À retenir : Rapport Signal sur Bruit : SNR 6.02 × B + 1.25 dB

Si16 bits 98 dB de dynamique, c'est énorme.

Codage PCM-Pulse Code Modulation

Modulation par Impulsions Codées , Inventé en 1937, c'est le standard absolu. Tout part de là : CD Audio, WAV, téléphonie. C'est du codage sans compression, on garde tout. Chaque échantillon est codé en binaire. Chaque niveau quantifié est converti en binaire. Le flux final est une suite de bits : 1000 1101 1111 1101...

— Débit binaire :

Débit = Fe x Nombre de bits x Nombre de canaux= F_e× B × C

C = nombre de canaux (1 mono, 2 stéréo)

Exemple CD : 44100 x 16 x 2 = 1 411 200 bit/s = 1,41 Mbit/s

— Taille d'un fichier :

Taille = $\frac{D\ x\ Duree}{8}$ (en octets) 1 minute de CD stéréo = 10,5 Mo. C'est pour ça qu'on compresse !

Évolutions du PCM :

— DPCM : On code la différence entre 2 échantillons (plus petite).
— ADPCM (G.721) : DPCM + pas adaptatif → 32 kbit/s.
— Modulation Delta : 1 seul bit : le signal monte ou descend ?

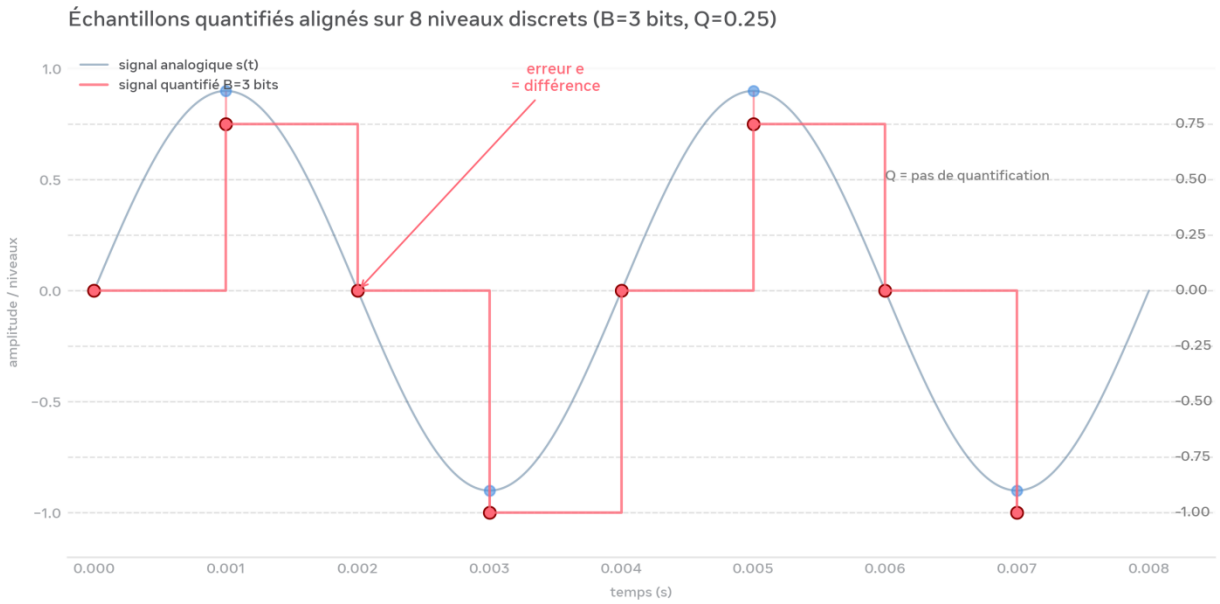


FIGURE 6 : Quantification

Compression audio

Le son brut est très lourd. On doit le compresser :

A. **Sans perte** : on retrouve exactement le son original en gardant le PCM exact : FLAC, ALAC, WAV : Réduction de 50%. Utilise la redondance.

Diviser par 2 sans rien perdre. Pour archivage studio.

— Free Lossless Audio Codec / ALAC : Prédiction linéaire : on prédit l'échantillon suivant à partir des précédents, on ne code que l'erreur de prédiction + Huffman.

Résultat : 1411 kbps (CD) → 700 kbps. Fichier 10,6 Mo → 5 Mo pour 1 min. Qualité 100% identique.

B. **Avec perte** : 90% de gain (psychoacoustique on supprime ce que l'oreille n'entend pas) irréversible mais on ne l'entend pas : MP3, AAC, OGG (Famille MPEG)

Basé sur la psychoacoustique : Notre oreille n'entend pas tout : effet de masquage. Un son fort cache un son faible proche en fréquence. On supprime ce que l'oreille ne percevra pas. La chaîne MP3 en 8 étapes :

1. **Entrée PCM** : 44,1kHz / 16bits / Stéréo = 1411 kbps
2. **Filtre en 32 sous-bandes** : On découpe le son en 32 bandes de fréquences (comme un égaliseur). Chaque bande = 625Hz de large.
3. **FFT** : Transformée de Fourier pour avoir le spectre précis de chaque bande.
4. **Modèle psychoacoustique** : Le codeur simule ton oreille :
5. **Masquage fréquentiel** : Si j'ai un son fort à 1000Hz à 60dB, tous les sons faibles entre 900-1100Hz à moins de 30dB sont inaudibles → on les supprime. Masquage temporel : Un gros son masque les sons 5ms avant et 100ms après. Le modèle calcule pour chaque bande le seuil de masquage.
6. **Seuil de masquage :** Tout ce qui est en dessous du seuil est supprimé. C'est là qu'on gagne 80% de la place.

Table 1 : Comparaison compression avec perte vs sans perte

Format	Débit	Taux	Qualité
PCM CD	1411 kbps	%	Référence
FLAC	∼700 kbps	∼50%	Identique
MP3 128k	128 kbps	∼91%	Bonne (YouTube)
MP3 320k	320 kbps	∼77%	Très bonne
AAC 128k	128 kbps	∼91%	Meilleure que MP3 128k

7. **Quantification & allocation des bits :** On ne donne pas le même nombre de bits à chaque bande. Bande très audible (2-4kHz voix) → on met 8 bits Bande masquée → on met 0 bit C'est adaptatif : le bitrate se balade là où c'est utile.
8. **Codage Huffman :** On code le tout avec Huffman (les valeurs fréquentes sur peu de bits).
9. **Trame MP3 :** Assemblage : Header (32 bits : synchro, débit, fréquence) + Données + CRC optionnel. 1 trame = 1152 échantillons = 26ms de son.

Famille MPEG Audio :

- **MP3** (MPEG-1 Layer III) : Découpe en 32 bandes → modèle psychoacoustique → supprime les parties masquées → quantifie. 128 kbit/s = qualité CD acceptable.
- **AAC** : Successeur MP3, 30% plus efficace. YouTube, iPhone.
- **Opus** : Le meilleur aujourd'hui pour la voix basse latence la voix temps réel . Combine SILK (voix) + CELT (musique). Débit de 6 kbps à 510 kbps, latence 5ms (WhatsApp, Discord, Zoom).
- **MIDI** : Ce N'EST PAS du son !0 kbps audio C'est une partition : "joue note 60 vitesse 100 pendant 500ms". Quelques Ko vs Mo.

Différence MP3 vs AAC : MP3 en 32 sous-bandes (précision moyenne). AAC remplace ça par une MDCT (Transformée Cosinus Modifiée) avec 1024 bandes : beaucoup plus précis, surtout dans les basses fréquences. + outils TNS, SBR. À 128k, AAC sonne comme MP3 à 192k.

VBR - Variable Bit Rate c'est l'astuce ultime du MP3 pour gagner encore 30% sans perdre en qualité.

1. CBR (Constant Bit Rate) Tu mets 128 kbps partout. Même pendant un silence, même pendant un solo de batterie ultra complexe. On gaspilles.

Débit = f_e×B ×Voies = fixe.
Exemple : 44100 × 16 × 2 = 1.4 Mbps tout le temps.

2. VBR - l'astuce : Le codeur analyse la complexit instantanée grâce au modèle psychoacoustique

- Passage simple / silence / son pur à 1000 Hz masqué -> il met 32 kbps seulement. Il sait que tu n'entendras pas la différence.
- Passage complexe / attaque de cymbale / orchestre complet -> il monte à 320 kbps pour ne rien rater.

Quand l'utiliser ?
CBR = streaming radio, il faut un débit constant pour ne pas couper
VBR = archivage, le disque dur, Spotify en qualité max. C'est toujours meilleur.

Les traitements numériques courants

une fois le son numérisé, on lui applique des traitements numériques avec des algorithmes (DSP) :

— **Filtrage** : Passe-bas (enlève les aigus), passe-haut (enlève les graves)
— **Égalisation** (EQ) : Augmenter/diminuer certaines fréquences
— **Réduction de bruit** / Écho / Réverbération : en supprimant le bruit de fond
— **Analyse** : FFT (Transformée de Fourier) pour passer du domaine temporel au domaine fréquentiel et afficher un spectrogramme. C'est la base de Shazam et de la reconnaissance vocale.

2.2 Image

Une image numérique est une représentation spatiale bidimensionnelle (2D) d'une scène visuelle, généralement structurée sous forme de matrice.

C'est une fonction 2D f(x,y) discrétisée. x,y = position du pixel, f = couleur/luminance.

Notions clés :

— **Pixel** : Plus petite unité de l'image.
— **Résolution** : Nombre de pixels (ex : 1920x1080) et densité (DPI/PPI).
— **Profondeur de couleur** : 1 bit (noir/blanc), 8 bits (256 couleurs), 24 bits (16,7 millions - True Color).
— **Modèles de couleur** : RGB (écrans), CMJN (impression), YUV (vidéo, sépare luminance et chrominance).

Types d'images

Donnée spatiale bidimensionnelle statique, caractérisée par sa résolution et son espace colorimétrique, on distingue 2 types entièrement différents :

— **Image Vectorielle :**Représentée par des primitives géométriques (points, lignes, courbes) définies par des équations mathématiques, permettant un redimensionnement sans perte de qualité.



FIGURE 7 : Image Vectorielle

Pas de pixels ! Des formules mathématiques : "trace un cercle de rayon 10 en (x,y) Une image vectorielle = pas de pixels, que des maths. Elle est construite à partir d'objets géométriques décrits par équations → zoom infini sans flou.

Les 5 primitives :

— Point : (x,y)

— Ligne : 2 points reliés
— Courbe Bézier : lisse contrôlée par points de contrôle (c'est la base de toutes les typos et logos)
— Polygone : n côtés
— Cercle/Ellipse : centre + rayon

Les 2 formules que vous devez retenir :

1. Droite / Segment :

$$P = P_0 + t(P_1 - P_0) \quad avec \quad t \in [0 - 1] \tag{1}$$

Tu fais varier t de 0 à 1 et ton point P glisse de P_0P_1.

2.Courbe de Bézier cubique : (la plus utilisée)

$$B(t) = (1-t)^3P_0 + 3(1-t)^2tP_1 + 3(1-t)t^2P_2 + t^3P_3 \tag{2}$$

4 points de contrôle

$$P_0, P_1, P_2, P_3, t \in [0 - 1]. \tag{3}$$

C'est avec ça qu'on dessine tout dans Illustrator.

Indépendante de la résolution : zoom infini sans perte. **Formats :** SVG, AI, EPS, Utilisée pour : logos, icônes, schémas

— **Image Matricielle (Bitmap) :**Composée d'une grille de pixels. Chaque pixel contient des informations de couleur codées selon un modèle (ex : RVB pour le rouge, vert, bleu). La résolution (nombre de pixels) et la profondeur de couleur (bits par pixel) déterminent la précision de l'image.

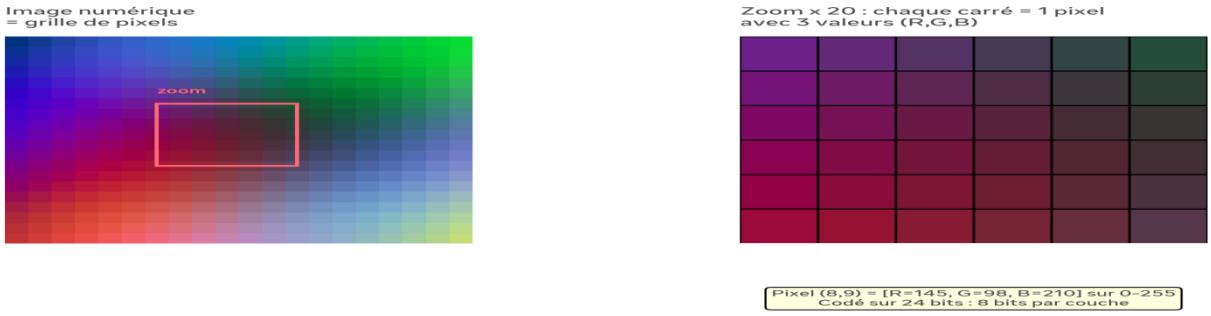


FIGURE 8 : Image Matricielle et son Zoom

La figure montre une grille de pixels colorés, avec un zoom sur une petite zone montrant les carrés individuels des pixels.

Formats : JPEG, PNG, BMP, TIFF, utilisée pour : photos, images réalistes

Types d'images matricielle :

a) **Noir et Blanc binaire** - 1 bit / pixel : Le plus simple.

1 pixel = 1 bit → 0 = noir, 1 = blanc 2 couleurs seulement.
Taille : Taille = L x H / 8 octets, Exemple 1920x1080 = 259 Ko
Usage : fax, QR code, dessin au trait.

b) **Niveaux de gris** : 8 bits / pixel

1 pixel = 8 bits → 256 niveaux de gris de 0 (noir) à 255 (blanc)
C'est la luminance pure, sans couleur.
Taille : Taille = L x H octets, Exemple 1920x1080 = 2 Mo
Usage : imagerie médicale (radio), photo noir et blanc, c'est le canal Y du YUV.

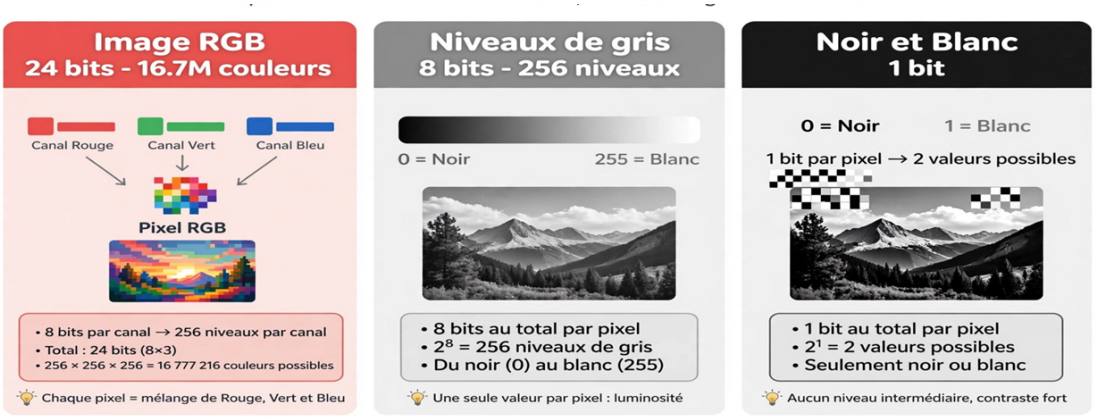


FIGURE 9 : Types d'image Matricielle

C) **Couleur RGB** - 24 bits / pixel

C'est le standard des écrans. Synthèse additive.

1. 1 pixel = 3 canaux × 8 bits = 24 bits
2. R = Rouge, G = Vert, B = Bleu, chaque canal de 0 à 255 Nombre de couleurs :2^{24} = 16 777 216 couleurs (True Color)

Taille brute : Taille = L x H x 3 octets, Exemple 1920x1080 = 6,22 Mo en brut sans compression ! Si on passe en 32 bits, les 8 bits supplémentaires c'est le canal Alpha (transparence) = RGBA.

Modèles de couleurs

— **RGB** : Pour les écrans (additif). R+G+B = Blanc.
— **CMJN (CMYK)** : Pour l'impression (soustractif). Cyan, Magenta, Jaune, Noir.
C+M+J = Noir théorique.
— **YUV / YCbCr** : Pour la vidéo et la compression JPEG/MPEG. C'est la révolution :
Y = Luminance (le détail que l'œil voit le plus) U,V = Chrominance (la couleur) L'œil est très sensible à Y et peu sensible à U,V. Donc on va compresser beaucoup plus la couleur que la luminance. C'est la base du sous-échantillonnage 4:2:0.
— **HSV** : Pour le traitement : Teinte, Saturation, Valeur. Plus intuitif pour l'humain.

Compression avec perte

JPEG - Comment on passe de 6 Mo à 200 Ko C'est la chaîne que vous devez savoir en 5 étapes :

1. **RGB vres YUV(YCbCr)** On sépare luminance et chrominance

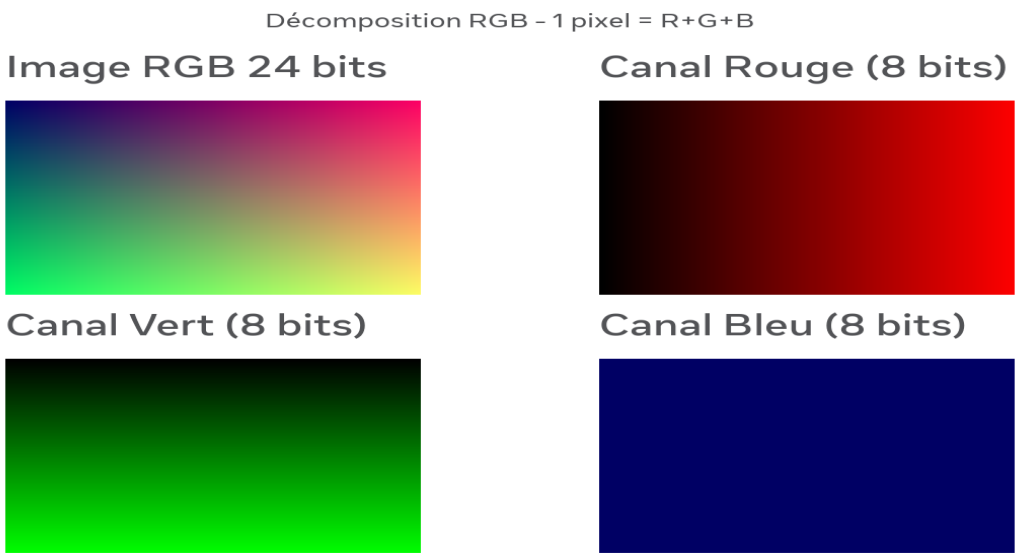


FIGURE 10 : Image RGB , canal rouge - Vert et Bleu

Est une conversion très simple, c'est juste un changement d'espace couleur pour pouvoir faire la compression par la suite . RVB = Rouge Vert Bleu, ce qu'on voit à l'écran.

YCbCr = ce que la vidéo et la compression JPEG utilisent :

Y = Luminance, la lumière / niveau de gris

Cb = Chrominance bleue (bleu - luminance)

Cr = Chrominance rouge (rouge - luminance)

On sépare la lumière de la couleur pour pouvoir compresser la couleur plus fort sans que l'oeil le voie.

Les 2 formules à connaître, selon si tu es en full-range [0-255] pour JPEG / OpenCV, ou en studio-range [16-235] pour la TV :

Standard BT.601 pour SD/JPEG Pour SD / JPEG full range [0-255], C'est celle qu'utilise cv2.cvtColor(img, cv2.COLOR_BGR2YCrCb) :

Avec R,G,B en :[0][1]

— Y = 0.299 R + 0.587 G + 0.114 B

— Cb = -0.1687 R - 0.3313 G + 0.5 B + 0.5

— Cr = 0.5 R - 0.4187 G - 0.0813 B + 0.5

Avec R,G,B en [0][255]-> Y,Cb,Cr en :[0][255]

— Y = 0.299 R + 0.587 G + 0.114 B

— Cb = 128 - 0.168736 R - 0.331264 G + 0.5 B

— Cr = 128 + 0.5 R - 0.418688 G - 0.081312 B

L' inverse :

— R = Y + 1.402 (Cr-128)

— G = Y - 0.344136 (Cb-128) - 0.714136 (Cr-128)

— B = Y + 1.772 (Cb-128)

Standard TV HD **BT.601 studio-range** [16-235][16-240] Utilisé en vidéo MPEG, H.264, etc.

— Y = 16 + 65.738 R + 129.057 G + 25.064 B
— Cb = 128 - 37.945 R - 74.494 G + 112.439 B
— Cr = 128 + 112.439 R - 94.154 G - 18.285 B

avec R,G,B normalisés en.[0][1]

BT.709 pour HD / BT.2020 pour 4K HDR, les coefficients changent légèrement pour Y :

BT.709 :

— Y = 0.2126 R + 0.7152 G + 0.0722 B

Sous-échantillonnage 4:2:0 : On garde Y en pleine résolution, mais on divise par 4 la résolution de Cb et Cr (on divise sur 2 horizontalement ET sur 2 verticalement) . On ne voit presque pas la différence, mais on gagne 50% de taille. C'est le fameux chroma subsampling :

4:4:4 = pas de compression, Y, Cb, Cr en pleine résolution

4:2:2 = Cb,Cr à moitié en largeur (utilisé en pro video), chroma divise par 2 horizontalement

4:2:0 = Cb,Cr divise par 2 la largeur ET hauteur. C'est ce que fait tout JPEG, tout H.264, H.265, YouTube, Netflix.

DCT : On ne fait pas la DCT sur toute l'image d'un coup, on la découpe en petits carrés de 8x8 pixels sur le canal Y qu'on a obtenu après YCbCr et on applique la Transformée en Cosinus Discrète. On passe de l'espace spatial à l'espace fréquentiel. On obtient 1 basse fréquence (l'image moyenne du bloc) + 63 hautes fréquences (les détails).

Pour un bloc 1D de 8 pixels x :[n] La DCT ne compresse toujours rien. Elle ne fait que réorganiser

$$F[k] = C(k) \sum_{n=0}^{7} x[n] \cos\left(\frac{\pi k(2n+1)}{16}\right)$$

FIGURE 11 : Equation DCT

Quantification :

~~3.~~ On divise chaque des 64 coefficients par un pas différent, issu d'une table de quantification.
— On divise très peu les basses fréquences (on les garde précises)
— On divise très fort les hautes fréquences (elles deviennent 0)
— Plus le facteur de qualité est bas, plus on écrase les détails et si Q est grand → image floue mais petite.

C'est l'étape irréversible avec perte .

Codage entropique : Balayage en Zigzag + RLE (on code les séries de zéros) + Huffman.

Résultat : 6 Mo → 300 Ko avec qualité 90% invisible à l'œil. PNG est sans perte : il utilise du LZ77 + Huffman, il garde tout.

120	15	-3	0	0	0	0	0
8	2	0	0	0	0	0	0
-1	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0

TABLE 2 : Exemple Image

Le Zigzag Si tu lis le bloc ligne par ligne, tu aurais : 120,15,-3,0,0,0... 8,2,0... avec des zéros entre les valeurs, nul.

Le zigzag sert à regrouper tous les zéros à la fin. On parcourt en diagonale, des basses fréquences vers les hautes. **Le codage : DC et AC séparés**

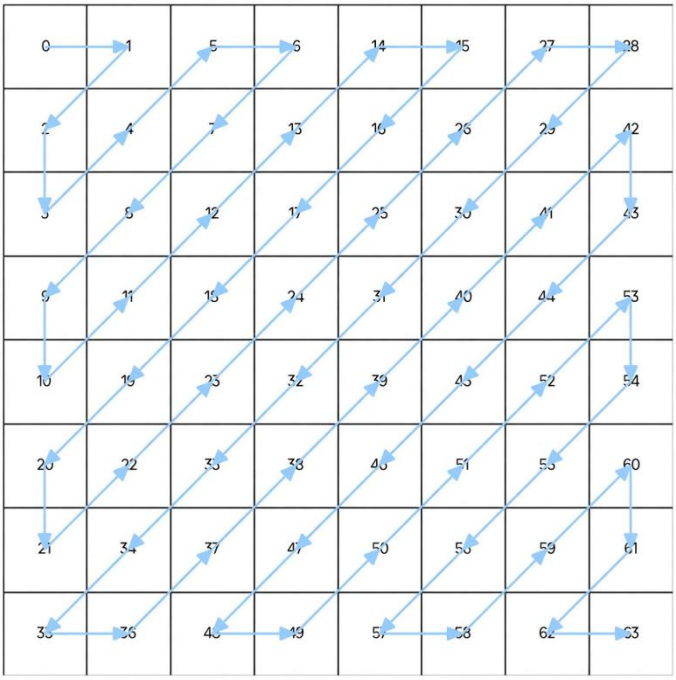


FIGURE 12 : Zigzag

On ne code pas les 64 valeurs de la même façon.

a) Le DC (position 0 = 120) : codage différentiel Le DC c'est la moyenne du bloc. D'un bloc au bloc voisin, elle change très peu. Donc on ne code pas 120, on code la différence avec le bloc précédent. Si le bloc précédent avait DC = 115 :Diff = 120 - 115 = 5 On code 5. Beaucoup plus petit = moins de bits. b) Les AC (positions 1 à 63) : RLE + Zigzag On code les 63 valeurs restantes avec des paires (nombre de zéros avant, valeur). Notre suite AC : 15, 8, -1, 2, -3, 0,0,0...

ça devient :

(0, 15) (0, 8) (0, -1) (0, 2) (0, -3) EOB : End Of Block.

On dit "tout le reste c'est des zéros, j'arrête".

Sans zigzag + RLE, on aurait dû coder 63 valeurs. Là on en code 5 + EOB : Gain énorme.

2.3 Vidéo

La vidéo est une succession d'images (frames) + du son On affiche une suite d'images fixes très rapidement. Comme vous le voyez sur la figure ci-dessous , les 5 Frames sont presque identiques (même paysage). C'est la persistance rétinienne : à partir de 24 images par seconde, notre cerveau ne voit plus des images distinctes, mais un mouvement fluide.

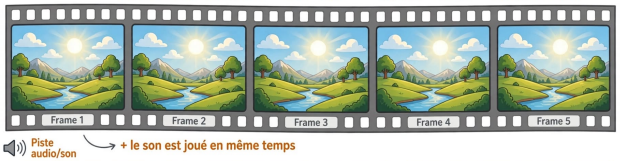


FIGURE 13 : Frames Video

b) Le son : En dessous de la pellicule, la flèche indique "Piste audio/son + le son est joué en même temps". Sans l'audio synchronisé, ce n'est qu'une animation. La vidéo, c'est image + son synchrones.

Qualité de la vidéo

Les 3 paramètres techniques qui définissent la qualité d'une vidéo sont :
1 - Résolution : Le nombre de pixels par image

1. 1280x720 = HD (921 600 pixels par image)
2. 1920x1080 = Full HD (2 millions de pixels)
3. 3840x2160 = 4K (8 millions de pixels)

Plus la résolution est grande, plus l'image est nette, mais plus elle pèse lourd. **2 - Fréquence d'images** : Frame Rate en fps c'est le nombre d'images par seconde.

1. 24 fps = standard cinéma
2. 30 fps = vidéo normale (téléphone, YouTube)
3. 60 fps = très fluide (jeux vidéo, sport)

C'est directement lié à la fluidité. **3- Codec de compression :** On a vu avec le RLE que sans compression une image 8x8 pèse 64 bits. Imagine une vidéo 4K à 60 fps : c'est impossible à stocker sans compression.

1. H.264 : le plus standard et compatible
2. H.265 / HEVC : compresse 2x mieux que H.264
3. VP9 / AV1 : les codecs modernes de YouTube/Netflix, poids très réduit.

Le but : "réduire la taille sans perdre la qualité" La formule finale Taille vidéo = Résolution x fps x Profondeur couleur x Durée

Si on augmente un seul facteur (résolution, fps, durée), le fichier grossit. Le codec est là pour compenser et diviser cette taille par 100 ou 200. En résumé : Si une image 8x8 c'est une photo, une vidéo 8x8 à 30 fps pendant 1 seconde, c'est 30 fois cette photo + une piste audio, le tout compressé par un codec.

Compression Vidéo :

Sans compression, une vidéo est impossible à utiliser. **Le problème de départ :** VIDÉO = DES MILLIERS D'IMAGES ! 1 min en 4K non compressée = environ 5000 images. En réalité une image 4K = 3840 x 2160 pixels x 3 couleurs x 8 bits = 25 Mo.

À 60 images/sec = 1,5 Go par seconde.

1 minute = 90 G 1 Go toutes les 12 secondes. Impossible à stocker ou à streamer. D'où les 2 types de compression que combinent tous les codecs modernes :

1. Compression INTRA-Image - à l'intérieur d'une image "Comme le JPEG, appliquée à chaque image" C'est ce qu'on a fait avec le RLE sur l' image 8x8. On supprime les détails invisibles à l'oeil (nuances de couleurs, textures). On compresse chaque frame indépendamment, comme si c'était une photo JPEG.

Gain : -70% de taille . Mais ça ne suffit pas, car 5000 JPEGs à la suite restent énormes.

2. Compression INTER-Images : entre les images

"On ne garde que les changements entre les images" C'est l'idée géniale de la vidéo. Regarde la pellicule Frame 1 à Frame 5 dans la figure précédente : 95% de l'image ne bouge pas. Le ciel, les montagnes restent identiques.

Donc au lieu de stocker 5 images complètes, on stocke : 1 image complète, puis seulement la différence ou le mouvement.

Compression H.264 / H.265 / AV1 C'est JPEG avec prédiction temporelle. Pour chaque bloc 8x8 ou 16x16 en passant par 4 étapes :

1. **Découpe en GOP (Group Of Pictures) :** I P B B P B B I ... Le GOP (Group Of Pictures) est une séquence de frames vidéo regroupées pour la compression. Il commence par une frame I et contient des frames P et B. C'est le coeur de la vidéo moderne. Un GOP est un groupe d'images traitées ensemble, ici 9 images = 0,5 seconde à 24 fps. Il y a 3 types d'images dans le GOP : - I-frame (Intra) : Image clé , complète stockée comme une photo JPEG. C'est la référence. C'est lourde mais indispensable pour commencer , encodée seule, sans dépendre d'autres images. C'est le point de départ de la vidéo. on applique la compression intra : YCbCr -> DCT -> Quantification -> Zigzag -> Huffman. 1 toutes les 1-2 secondes. **A quoiça sert ? :** Seek / Navigation : quand tu cliques au milieu d'une vidéo YouTube, le lecteur saute à la I-Frame la plus proche pour afficher instantanément. Elle est indépendante, pas besoin d'images précédentes. C'est la base qualité pour la suite. Les P-Frame et B-Frame qui suivent vont se baser dessus.

   Sans I-Frame, tu ne pourrais jamais démarrer une vidéo au milieu.

   - P-frame (Prédictive) : ne stocke que les différences par rapport à la frame précédente grâce à la prédiction du mouvement. - B-frame (Bidirectionnelle) : Encore plus intelligente, elle stocke le mouvement par rapport à l'image d'avant ET d'après. C'est la plus légère mais nécessite un délai . Séquence typique sur le schéma : I - P - B - B - B - B - I ...etc.

   B-frame est la compression la plus élevée des trois types (I, P, B), Moins de données stockées / moins d'octets elle est Idéale pour réduire le débit binaire, C'est pour ça qu'on en met beaucoup dans une vidéo. Dans un GOP typique I B B P B B, les B sont majoritaires.

   Elle nécessite un Buffer (Mémoire tampon) - LE POINT CRITIQUE car la future image (t+1) doit être décodée avant la B-Frame Entraîne un délai / retard de décodage Le décodeur stocke

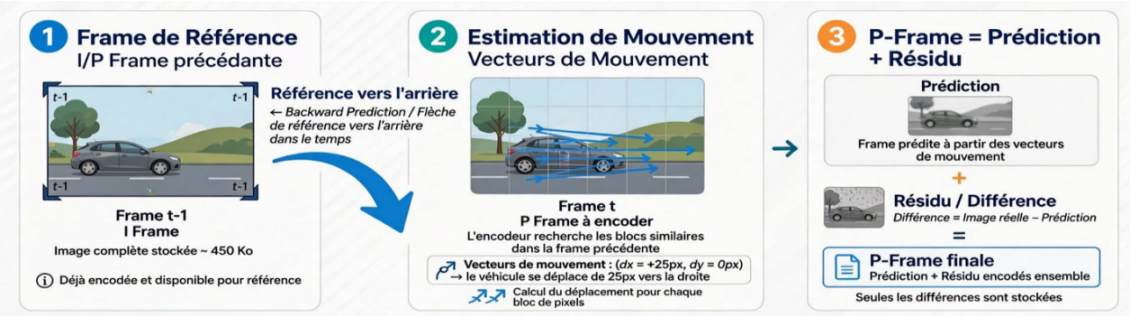


FIGURE 14 : Image Prédictive

temporairement les frames en mémoire Pourquoi ? Paradoxal : pour décoder l'image du présent (t), il te faut l'image du futur (t+1). Donc le décodeur doit d'abord recevoir et garder t+1 en mémoire, puis revenir à t. Ça crée une latence. C'est pour ça que la visioconférence (Zoom) utilise peu de B-Frames : ça ajoute du délai. Le décodeur stocke temporairement les frames en mémoire Le GOP recommence

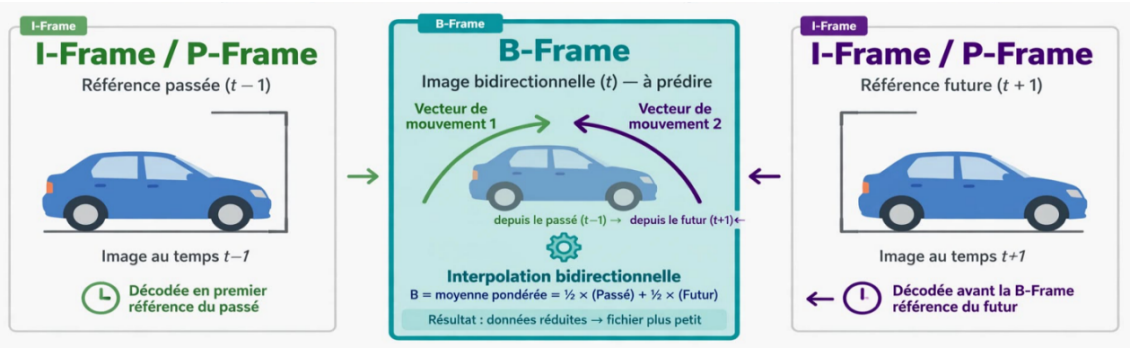


FIGURE 15 : Image Bidirectionnelle

avec une nouvelle image I. Si tu perds le signal, tu attends la prochaine I pour reprendre.

2. **Estimation de mouvement**

   Pour chaque bloc 16x16 de l'image actuelle, l'encodeur cherche le bloc le plus ressemblant dans l'image précédente.

   Il trouve un vecteur de mouvement : MV = (dx, dy) par exemple (-3, +1) Il ne code que ce vecteur : 2 petits nombres au lieu de 256 pixels. Gain énorme.

   Découpage en macroblocs 16x16 La trame courante est divisée en macroblocs de 16x16 pixels. Chaque bloc est traité séparément. l'image complète est découpée comme une grille. On ne cherche pas le mouvement de l'image entière, on cherche le mouvement de chaque petit carré de 16x16. Pourquoi 16x16 ? C'est le compromis historique MPEG / H.264 / H.265. Assez grand pour être efficace, assez petit pour suivre un objet qui bouge. Sur le schéma, le macrobloc à coder (16x16) est encadré en rouge avec un petit nuage dedans.

   Recherche dans la trame de référence On recherche le bloc le plus semblable dans la trame de référence précédente, dans une fenêtre de recherche.

- Trame de Référence (image précédente) : la I-Frame. - Fenêtre de recherche (±8 px) : on ne cherche pas dans toute l'image (trop coûteux). On cherche autour de la même position, dans un petit carré en pointillé. - Meilleur match trouvé : On teste plusieurs positions candidates dans la fenêtre. Déplacement du bloc. On a trouvé où est parti le nuage.

Vecteur de Mouvement (dx, dy) Le décalage entre la position du bloc dans la trame courante et la position du meilleur match est le vecteur de mouvement.

Position initiale (x,y) +4 px -> dx = +4 (vers la droite) -2 px -> dy = -2 (vers le bas)

Vecteur de mouvement : (dx, dy) = (+4, -2) pixels Le bloc s'est déplacé de 4 pixels vers la droite et 2 pixels vers le bas.

Au lieu de coder 16x16 = 256 pixels, on code juste (4, -2) = 2 petits nombres.

Calcul du SAD - Somme des Différences Absolues Comment on sait que c'est le "meilleur match" ? le critère mathématique : Formule :

$$SAD = \sum |I_{\text{courant}}(i,j) - I_{\text{référence}}(i,j)| \tag{4}$$

On fait la différence pixel par pixel en valeur absolue, et on additionne. L'exemple chiffré 4x4 (simplifié) : Bloc Courant : 52, 50, 48, 47 ;49,51,50,49 ;50,49,51,50 ;48,50,49,51 Bloc Référence : 50, 50, 48, 47 ;49,49,50,49 ;50,48,51,50 ;48,49,49,51 Différences absolues : |52-50|=2, |50-50|=0, etc.

SAD = 2+0+0+0+0+2+0+0+1+0+0+0+0+0+0+1 = 6 SAD = 6 -> meilleur match (valeur minimale) Plus le SAD est faible, plus les blocs sont similaires. Dans la pratique, on cherche le minimum du SAD sur toute la fenêtre pour trouver le meilleur vecteur de mouvement.

l' avantage est qu'on encode seulement le vecteur (dx,dy) + le résidu, au lieu du bloc complet.

Critère SAD : rapide et simple, largement utilisé (MPEG-2 / H.264 / H.265 / HEVC) Son objectif est de réduire la quantité de données à coder -> gain de compression C'est pour ça qu'une P-Frame de 42 Ko suffit alors qu'une I-Frame fait 450 Ko.

Le résidu La prédiction n'est jamais parfaite. Il reste une différence :

Résidu = Bloc réel - Bloc prédit (déplacé)

Ce résidu ressemble à une image très sombre, presque noire, avec juste les contours qui ont bougé. On le compresse avec exactement ce qu'on a vu : DCT + Quantification JPEG + Zigzag + Huffman.

Compression Sans Perte : la compression sans perte - Encodage Lossless suit la chaîne suivante : Image originale → Prédiction spatiale/temporelle (on prédit le pixel d'après) → Résidu (différence) → Codage entropique CABAC/Golomb/RLE → Bitstream → Reconstruction 100% identique

Pas de DCT, pas de quantification → aucune info supprimée

PSNR = Infini, fidélité parfaite Formats : FFV1, HuffYUV, ProRes 4444 XQ, H.264 lossless, H.265 lossless

Débit 1080p : 200-400 Mbps → énorme, 1 min = 3 Go

GOP : I → P → P → P → chaque image P est prédite mais reconstruite exactement

Quand l'utiliser ? Montage intermédiaire (éviter accumulation de pertes),archivage, médical et cinéma

Compression Avec Perte : la compression avec perte - Encodage Lossy suit la chaîne suivante : Image → Estimation de mouvement (vecteurs) → Compensation → Résidu → DCT → Quantification forte (c'est ici qu'on perd) → Codage entropique + filtre deblocking → Bitstream → Reconstruction approchée DCT + Quantification = cœur de la perte, on supprime hautes fréquences peu visibles

PSNR = 30-45 dB, perte maîtrisée, visuellement imperceptible si bitrate correct

Formats : H.264/AVC, H.265/HEVC, VP9, AV1 Débit 1080p : 4-8 Mbps → 1 min = 30-60 Mo, ratio 50:1 à 200:1 GOP : I → B → P → B → P → I → I = clé DCT quantifiée, P = prédite en avant, B = bi-directionnelle (passé+futur)

Décompression Vidéo

On passe de l'encodage au décodage. 3.2 c'est l'inverse de tout ce qu'on a vu . C'est ce que fait ton téléphone quand il lit une vidéo H.264. Cette partierésume le chemin complet d'un fichier .mp4 jusqu'à l'image à l'écran suivant 5 étapess du décodage :

1. Décodage CABAC entropique Décodage binaire adaptatif par contexte - Reconstruction des coefficients quantifiés. C'est l'inverse du RLE/Huffman. Le fichier ne contient plus des pixels, mais un flux de bits 0101 0101. Le décodeur CABAC décompresse sans perte pour retrouver les coefficients + les vecteurs de mouvement (dx, dy). Sans cette étape, rien n'est lisible.

2. IDCT déquantification l'inverse résiduel : Transformation inverse DCT pour passer du domaine fréquentiel au spatial. On a vu en I-Frame : on avait transformé les blocs 8x8 en fréquences + quantifié (perte de détails). Ici on fait l'inverse :

Déquantification : on multiplie par la matrice Q pour retrouver les coefficients résiduels

IDCT : on repasse des fréquences aux pixels. On retrouve le résidu pixel, l'erreur de prédiction.

3. Compensation de mouvement inverse : Reconstruction des images P et B à partir de l'image I de référence. Utilisation des vecteurs de mouvement (MV) pour prédire le bloc

Interpolation des pixels selon les déplacements En clair : Image finale = Prédiction (déplacée par le vecteur) + Résidu (IDCT) Pour une I-Frame, cette étape est sautée. Pour une P-Frame on utilise une référence arrière, pour une B-Frame on moyenne arrière + avant (t-1 et t+1).

Conversion YCbCr 4:2:0 -> RVB : Upsampling de la chrominance + Conversion espace couleur. On avait converti RVB -> YCbCr 4:2:0 pour compresser. La chrominance Cb/Cr était sous-échantillonnée 4x plus petite que Y. Ici on fait l'inverse : **Upsampling Cb/Cr 4:2:0 -> 4:4:4** on redonne la même taille **Conversion : YCbCr -> RVB** pour l'affichage écran. Ton écran ne comprend que le RVB. **Interpolation temporelle et lissage** Lissage temporel pour réduire les artefacts. Dernière finition : on lisse entre frames pour fluidifier et filtrer pour stabiliser la vidéo et réduire le scintillement.

En résumé : Décoder = 1) Décompresser le flux 0101 2) Retrouver les fréquences 3) Redéplacer les blocs avec les vecteurs (dx,dy) 4) repasser en RVB -> 5) lisser et synchroniser le son avec PTS/DTS.

Lois et Mécanismes

Ce ne sont pas des étapes de décodage pixel, mais les lois qui font que la vidéo est regardable.

Synchronisation audio-vidéo : PTS DTS (Presentation Time Stamp / Decoding Time Stamp) : Synchronisation des horloges pour playback fluide. Le son ne doit pas être en retard sur l'image. DTS = quand décoder, PTS = quand afficher. Essentiel à cause des B-Frames qui se décodent dans le désordre.

Buffer : gestion du buffer vidéo pour anticiper les variations de débit. Éviter underflow (plus rien à afficher) / overflow (trop de données).

Jitter : variation temporelle des paquets. Compensé par buffer déjitter et horloge PLL.

Filtre deblocking H.264 : filtre en boucle anti-blocs. C'est la signature du H.264. Lissage des bords de macroblocs 4x4/8x8. Sans ça tu verrais la grille 16x16 de la 5/6. Réduction des effets de bloc.

QUALITÉ VIDÉO :

Comment on mesure si le décodage est bon ?

PSNR : Peak Signal-to-Noise Ratio - Mesuré en dB : 40 dB : excellente
30-40 dB : bonne
<30 dB : dégradée

SSIM - Structural Similarity Index - 0 à 1 (1 = identique) - Évalue structure, luminance, contraste : 0.95 : très haute qualité Standard : H.264 / AVC - Traitement temps réel - Optimisation débit / qualité

2.4 Haptique

La technologie haptique concerne la simulation du sens du toucher (forces, vibrations, textures) dans les interfaces numériques.

Est une donnée bidirectionnelle (interaction), impliquant des vecteurs de force et des signaux vibratoires synchronisés avec les flux visuels et sonores.

Norme IEEE 1918.1.1-2024 : definit architecture Tactile Internet, codecs haptiques et metriques QoE. Contrairement à audio/video unidirectionnels, l' haptique impose stabilite (passivite) et coherence temporelle.

Définition :

Du grec haptikos = "capable de toucher" ou Du grec haptomai = « je touche ». C'est l'ensemble des technologies qui permettent l'interaction par le sens du toucher. C'est le média le plus récent du multimédia.

C'est l'ensemble des techniques qui permettent d'échanger de l'information par le toucher. Si le son et l'image sont en sortie (tu vois et tu entends), l'haptique est en entrée ET sortie : tu touches ET tu ressens.

Composantes

On le sépare en 2 canaux totalement différents :

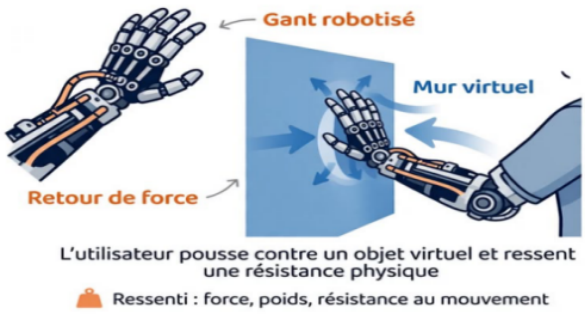


FIGURE 16 : Haptique Kinésthésique

Retour de force(Kinesthésique) : Représente les forces, les résistances et le poids et position des articulations. Il nécessite des capteurs et actionneurs pour simuler la sensation de manipuler un objet physique.

L'utilisateur pousse contre un objet virtuel et ressent une résistance physique. Schéma du Mur virtuel : la main robot essaie de traverser le mur bleu, le système bloque le mouvement et renvoie une force opposée.

Ressenti : force, poids, résistance au mouvement C'est beaucoup plus complexe. Il faut des moteurs qui peuvent bloquer ton bras.

Applications VR haut de gamme, téléchirurgie, télérobotique : tu sens le poids d'une pierre virtuelle. (proprioceptive) - ce que sentent les muscles et les tendons Techno : moteurs puissants, freins, exosquelettes

Exemples : volant de simulation qui devient dur dans un virage, joystick qui te repousse quand tu touches un mur virtuel, gant chirurgical qui te fait sentir la résistance d'un organe

Retour tactile : (cutanée) - ce que sent la peau Concerne la texture, la température ou les vibrations perçues par la peau, corpuscules de Meissner et Merkel au bout des doigts (ex : vibrations d'un smartphone). Techno : petit moteur ERM ou LRA qui vibre



FIGURE 17 : Haptique Tactile

Exemples : vibration du téléphone, retour haptique du clavier iPhone, manette PS5 qui crépite quand il pleut dans le jeu

Les données haptiques sont souvent échantillonnées à des fréquences très élevées (généralement 1 kHz ou plus) pour assurer une réactivité instantanée et éviter toute instabilité de perception.

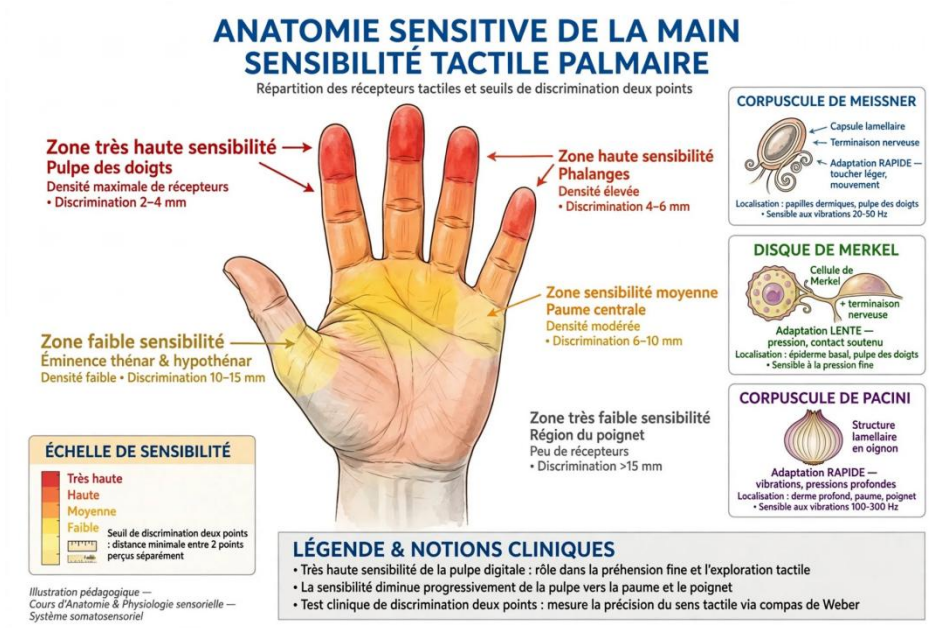


FIGURE 18 : Anatomie Sensitive de la main

Comment le traiter

C'est le coeur temps réel, commun aux deux types. C'est une boucle fermée qui tourne très vite : Capteur -> Moteur physique -> Actionneur La boucle haptique est une boucle fermée en temps réel à 1000 Hz minimum (sinon le cerveau sent le lag) :

Capteur : On capte et lire la position de la main / doigt ou la force de l'utilisateur.Détecte le toucher, la pression, le mouvement. Où est le doigt ? Quelle force ?

Moteur physique(Modélisation) : calcule la collision avec le monde virtuel. Ex : on a traversé un mur ? Quelle force je dois renvoyer ? Loi de Hooke $F = k \times x$ qui relie l'effort exercé sur un matériau élastique (comme un ressort) à sa déformation

F : Force ou tension en newtons (N)

k : Constante de raideur (ou de rappel) en newtons par mètre (N/m)

x (ou Delta l) : Allongement ou compression par rapport à la position d'équilibre en mètres (m) . Donc on Calcule la réponse en temps réel. Si le doigt est sur du sable, quelle vibration ? Si la main touche un mur, quelle force opposer ?

Actionneur : On restitue la force via un moteur, un gant haptique, une manette.

Cette boucle se répète des centaines de fois par seconde pour un ressenti haptique réaliste. Si elle est trop lente (>10ms), ton cerveau sent le décalage et l'illusion casse. Il faut <1ms pour le kinesthésique.

Compression Haptique

on passe de la technologie haptique (ce que tu ressens) au codage haptique (comment on le compresse pour l'envoyer).

Types de Codecs Haptique

Il existe 4 types :

(a) Tactile : Contact direct avec la surface. Tu touches, ça vibre. C'est binaire : touché / non touché.

2. Vibrotactile : Stimulations vibratoires haute fréquence. Comme le moteur de vibration du téléphone, mais à 200-300 Hz pour simuler une texture rugueuse.
3. Kinesthésique : Position, mouvement, force articulaire. Où est ta main dans l'espace ? Quel angle a ton coude ?
4. Force : Retour de force et résistance. C'est le mur virtuel de l'image précédente, tu pousses et ça résiste.

Codecs haptiques IEEE 1918.1.1

Publié pour la première fois le 14 juin 2024 après 8 ans de standardisation sous la direction du Prof. Eckehard Steinbach (TUM), c'est le premier codec standard pour le Tactile Internet. Il a été développé par un consortium TUM, Imperial College London, NYU Abu Dhabi, Dalian University et DLR

Codec kinesthesique sans delai

Prediction + Weber. Si délai réseau < 5ms, transmission directe 1 kHz, deadband perception (JND 5-15%).il enregistre les sensations pour les mouvements, à savoir les positions des membres et les forces agissant sur eux. Nombre de degrés de liberté (DoF), workspace.

Codec kinesthesique robuste au delai

Modele ressort-amortisseur virtuel, encodage energie, approche TDPA (Time Domain Passivity). Tolere 20-50ms.

protocole standardisé pour échanger les propriétés des devices lors de l'établissement de connexion : workspace, nombre de DoF, amplitude range, résolutions temporelle/spatiale

Codec tactile (vibrotactile)

La sensibilité de la peau, pour ressentir papier ou métal par exemple. Vibrations, textures

Extraction spectrale : 1 kHz vers coeffs (amplitude, frequence). Base sur modele Pacini, debit 5-20 kbit/s.

Etapes de Codage Haptique :

C'est exactement la chaîne de codage vidéo : échantillonnage -> quantification -> modélisation -> compression.

1. Échantillonnage haptique Fe = 1000 Hz (kinesthésique) - Loi de Shannon : Fe 2 x Fmax - Perception humaine : jusqu'à 500 Hz Comme en vidéo on avait 24 fps, ici il faut 1000 échantillons par seconde ! Pourquoi ? La main humaine perçoit jusqu'à 500 Hz (loi de Shannon : il faut échantillonner au moins 2x plus vite que la fréquence max). Si tu es à 100 Hz, tu sens le lag.

2. Quantification de la force 12-16 bits

Loi de Weber : L'humain ne sent une différence que si elle est >7-10%. Si tu portes 100g, il faut ajouter 10g pour sentir la différence. En dessous, on peut supprimer.

Loi de Stevens : La perception n'est pas linéaire, c'est une puissance.

$$\text{Perception} \approx \text{Intensité}^{\alpha} \tag{5}$$

On code la force sur 12 à 16 bits. Mais on applique la psycho-physique, comme on a supprimé les hautes fréquences invisibles en JPEG

3. Modélisation : Texture, Raideur, Frottement, Masse - Caractérisation des propriétés physiques de la surface.

On ne code plus une force brute, on code les paramètres physiques, comme on codait un vecteur de mouvement (dx, dy) au lieu des pixels : Texture, Raideur (ressort), Frottement, Masse.

4. Compression codecs haptiques - HAPTIC JPEB : compression adaptée au haptique - Deadband compression : suppression des valeurs < seuil JND C'est l'équivalent du H.264 pour la vidéo : - Deadband compression : Le plus utilisé. Si la variation de force est < JND (7%), on n'envoie rien.

Décodage Haptique :

comment on repasse du fichier binaire compressé à la vraie sensation dans ta main. Le processus passe par 4 étapes :

01. Décodage entropique - deadband inverse

- Reconstruction du flux binaire compressé
- Deadband inverse :seuillage adaptatif pour récupérer signaux faibles (évite bruit)

  On supprimait tout ce qui était < JND (7%). Ici on fait l'inverse, on reconstruit. Si on ne fait pas attention, on reconstruit du bruit. D'où le seuillage adaptatif.

  C'est l'équivalent du CABAC entropique en vidéo 3.2, on retrouve les bits 011111 0101100 en flux exploitable.

02. Déquantification - reconstruction force et texture

- Mapping niveaux quantifiés -> force et micro-textures[N]
- Interpolation : lissage pour surface/tactile continu on avait quantifié sur 12-16 bits. Ici on repasse de 1023 -> 2.5 Newton. Et on lisse, car si tu passes de 0 à 2.5N d'un coup, tu sens un à-coup. Interpolation = lissage pour surface continue.

03. Commande moteur haptique - loi contrôle impédance/admittance

-
- Formule centrale : F = K x + B v
- K = raideur (N/m), B = amortissement (N · s/m)
- Impédance : impose force en réponse au déplacement

— Admittance : impose mouvement en réponse à la force C'est le moteur physique de l'image haptique précédente.

Tu touches un mur virtuel :

— Impédance : je mesure où est ta main (x) et à quelle vitesse (v), je calcule quelle force F je dois te renvoyer pour simuler un mur raide K. C'est le plus courant.

— Admittance : je mesure ta force, et je bouge le bras robot en conséquence. Pour les gros exosquelettes. K = à quel point c'est dur, B = à quel point c'est visqueux.

04. Rendu moteur vibreur : ERM / LRA - Exosquelette - Filtre anti-vibration

— ERM : moteur excentrique, vibrations basse fréquence - cheap, lent, dans tous les téléphones

— LRA : Linear Resonant Actuator, réponse rapide de haute fréquence - précis, pour textures fines

— Exosquelette : restitution force multi-articulation

— Filtre anti-vibration : Notch/FIR pour supprimer résonances parasites

Sans filtre Notch, ton gant se met à vibrer tout seul à sa fréquence de résonance.

Lois et Critéres Clés

C'est l'équivalent des lois PTS/DTS et deblocking H.264

— Stabilité - Critère de passivité

Le système ne génère pas d'énergie : dissipatif - Condition : ReZ(s) 0 pour stabilité interactive

Loi fondamentale : ton gant haptique ne doit jamais te donner de l'énergie, sinon il s'emballe et te casse le doigt. Il doit toujours être dissipatif. Condition mathématique : partie réelle de l'impédance Z(s) 0.

— Z-width - Espace d'impédance rendable

Zone où l'impédance perçue est stable et réaliste - Dépend de la raideur max et amortissement minimal

C'est la plage entre le plus mou et le plus dur que ton système peut afficher sans devenir instable. Plus ton hardware est bon, plus le Z-width est grand. Un petit LRA de téléphone a un Z-width minuscule.

— Bande passante : 0-500 Hz

Perceptions tactiles 0-500 Hz exploitables - >500 Hz : atténuation, rendu texture fine limité

L'humain sent jusqu'à 500 Hz. Au-delà, on atténue. Comme en vidéo où l'oeil ne voit pas au-delà de certaines hautes fréquences.

— Latence < 30 ms - Loi de transparence Délai total boucle < 30 ms pour transparence - Objectif : ressentir l'environnement distant sans retard

En vidéo, on tolérait 100-200ms de buffer. En haptique, si tu as 30ms de retard entre le moment où tu touches et la force, tu sens que c'est fake et tu as la nausée. Pour le kinesthésique, on vise même <1ms en boucle locale

— Qualité haptique

- PSNR haptique : $10 \cdot \log10(\text{Psignal} / \text{Perreur})$ en dB -

-JND : Just Noticeable Difference 7-10% de la force Comme le PSNR vidéo : >40dB excellent. Et la JND de Weber de 4.1 : en dessous de 7-10% de changement, l'humain ne sent pas la différence, donc on peut compresser.

Applications

Médecine : Télé-chirurgie, formation de dentiste sur mannequin virtuel **Jeu vidéo** / VR : Le métavers sans haptique n'a aucun sens

Accessibilité : Plage braille, téléphone qui traduit le son en vibration pour les sourds

Industrie : Former un pilote à sentir les turbulences sans risquer un avion

Exemples d'application Exercice 1 :

Un son de 3 minutes enregistré en qualité CD : Sample Rate 44,1 kHz, Bit Depth 16 bits, Stéréo (2 canaux).

Qu'elle est la taille du son avant et aprés compression ?

Exercice 2 :

Taille d'une image (Raster) Une image fait 1920x1080 pixels en True Color 24 bits. a) Calcule sa taille non compressée en Mo.

b) Si on la compresse en JPEG à 10%, quelle est la nouvelle taille ?

Exercice 3 :

On considère une image numérique en noir et blanc (1 bit par pixel : 0 pour le blanc, 1 pour le noir) de dimensions 8×8 pixels. La première ligne de cette image présente la séquence de pixels suivante : 0 0 0 0 1 1 0 0.

Questions :

1. Calculer le poids (en bits) d'une seule ligne de 8 pixels sans compression.
2. Calculer le poids total de l'image de 8 lignes sans compression (en bits et en octets).
3. Appliquer une compression de type RLE sur la séquence de cette première ligne.
4. Calculer le nombre d'éléments du flux compressé et commenter l'efficacité.

Conclusion

Ce premier chapitre nous a permis de mettre en évidence que la notion de multimédia ne recouvre pas une réalité homogène, mais un ensemble de signaux de nature profondément différente, tant par leur dimensionnalité que par leur interaction avec le système perceptif humain.

Bien que les techniques diffèrent, tous les médias obéissent au même principe : réduire le débit binaire en exploitant à la fois la redondance statistique du signal et l'imperfection des sens humains.

Maintenant que nos médias sont codés (I/P/B, CABAC, Deadband), ils doivent voyager ensemble sur le même réseau. Cependant, compresser efficacement ne suffit pas. Une fois réduits, ces flux hétérogènes, avec des débits, des tailles et surtout des contraintes temporelles radicalement différentes, doivent coexister sur un même réseau.

C'est ce qui pose la problématique du chapitre suivant : comment transmettre de manière synchronisée, avec une qualité de service garantie, des médias qui n'ont ni la même nature ni les mêmes exigences ?

Le Chapitre 2 sera donc consacré aux principes et contraintes de la transmission multimédia : bande passante, latence, gigue, pertes, synchronisation intra et inter-média, et aux architectures de transport et protocoles [RTP/RTCP, DASH] permettant d'y répondre.